

Joint Admission Control and Resource Allocation of Virtual Network Embedding via Hierarchical Deep Reinforcement Learning

Tianfu Wang¹, Li Shen², Qilin Fan³, *Member, IEEE*, Tong Xu⁴, *Member, IEEE*,
Tongliang Liu⁵, *Senior Member, IEEE*, and Hui Xiong⁶, *Fellow, IEEE*

Abstract—As an essential resource management problem in network virtualization, virtual network embedding (VNE) aims to allocate the finite resources of physical network to sequentially arriving virtual network requests (VNRs) with different resource demands. Since this is an NP-hard combinatorial optimization problem, many efforts have been made to provide viable solutions. However, most existing approaches have either ignored the admission control of VNRs, which has a potential impact on long-term performances, or not fully exploited the temporal and topological features of the physical network and VNRs. In this article, we propose a deep Hierarchical Reinforcement Learning approach to learn a joint Admission Control and Resource Allocation policy for VNE, named HRL-ACRA. Specifically, the whole VNE process is decomposed into an upper-level policy for deciding whether to admit the arriving VNR or not and a lower-level policy for allocating resources of the physical network to meet the requirement of VNR through the HRL approach. Considering the proximal policy optimization as the basic training algorithm, we also adopt the average reward method to address the infinite horizon problem of the upper-level agent and design a customized multi-objective intrinsic reward to alleviate the sparse reward issue of the lower-level agent. Moreover, we develop a deep feature-aware graph neural network to capture the features of VNR and physical network and exploit a sequence-to-sequence model to generate embedding actions iteratively. Finally, extensive experiments are conducted in various

settings, and show that HRL-ACRA outperforms state-of-the-art baselines in terms of both the acceptance ratio and long-term average revenue.

Index Terms—Network virtualization, virtual network embedding, deep reinforcement learning.

I. INTRODUCTION

AS THE demand for Quality of Service (QoS) has been continuously strengthened, research on efficient network resource management architectures has received massive attention from academia and industry. Traditional network management architectures directly execute diverse network functions on different dedicated servers, confronted with the increasing pressure of network services. Network virtualization has emerged as one of the promising approaches to overcome this problem, which is capable of decoupling the network services from their underlying hardware and empowering the programmability of services [1]. By integrating advanced technologies such as software-defined networking (SDN) and network function virtualization (NFV), network virtualization has become an essential part of next-generation networks for its superior capabilities to improve network resource utilization and reduce network difficulty management significantly [2].

In the network virtualization framework, user network requests are constructed as graph-structured virtual network requests (VNRs), consisting of virtual network functions and virtual links, dynamically arriving at the physical network to acquire resources. Then, the Internet providers will attempt to allocate node and link resources of physical network for VNRs under various constraints. To maximize the revenue of Internet providers, we need to decide which VNRs to be accepted and how resources to be allocated, constituting an NP-hard combinatorial optimization problem, called virtual network embedding (VNE) [3].

For this fundamental network management problem, many research efforts have been devoted to improving the performance of VNE. Classical approaches are categorized as the exact [4], [5], [6], the heuristic [7], [8], [9] and the meta-heuristic [10], [11], [12] ones. For exact algorithms, they aim at finding the optimal embedding solutions yet result in high computational complexity. Heuristic methods require rich expert knowledge and trial experience to design efficient heuristics. At the same

Manuscript received 16 March 2023; revised 2 October 2023; accepted 11 October 2023. Date of publication 23 October 2023; date of current version 12 June 2024. This work was supported in part by the National Natural Science Foundation of China under Grant 92370204, and in part by the Science and Technology Planning Project of Guangdong Province under Grant 2023A0505050111. Recommended for acceptance by Y. Wang. (*Corresponding authors: Hui Xiong and Li Shen.*)

Tianfu Wang and Tong Xu are with the Anhui Province Key Laboratory of Big Data Analysis and Application, School of Computer Science, University of Science and Technology of China, Hefei 230027, China (e-mail: tianfuwang@mail.ustc.edu.cn; tongxu@ustc.edu.cn).

Qilin Fan is with the School of Big Data and Software Engineering, Chongqing University, Chongqing 401331, China (e-mail: fanqilin@cqu.edu.cn).

Li Shen is with the JD Explore Academy, Beijing 101111, China (e-mail: mathshenli@gmail.com).

Tongliang Liu is with the UBTECH Sydney Artificial Intelligence Centre and the School of Information Technologies, Faculty of Engineering and Information Technologies, The University of Sydney, Darlingtown, NSW 2008, Australia (e-mail: tongliang.liu@sydney.edu.au).

Hui Xiong is with the Hong Kong University of Science and Technology (Guangzhou) Thrust of Artificial Intelligence Nansha, Guangzhou, Guangdong 511400, China, and also with the Department of Computer Science and Engineering, The Hong Kong University of Science and Technology, Hong Kong, SAR, China (e-mail: xionghui@ust.hk).

Our code is available at <https://github.com/GeminiLight/hrl-acra>.

This article has supplementary downloadable material available at <https://doi.org/10.1109/TSC.2023.3326539>, provided by the authors.

Digital Object Identifier 10.1109/TSC.2023.3326539

time, meta-heuristic approaches are often non-deterministic and depend on extensive searches to discover high-quality solutions.

Recently, several learning-based approaches [13], [14], [15] have been proposed due to their excellent abilities to build better heuristics from data automatically. However, these works pose two challenges. First, they rely on manually-extracted features or merely exploit partial latent temporal or topological features of the physical network and VNRs. Insufficient representation capability will limit the accuracy of VNE decisions. Second, most existing approaches can only optimize resource allocation for the current incoming VNR, ignoring the admission control of VNRs which has a potential impact on long-term benefits. VNRs need to compete for the limited resources provided by the physical network. An embedded VNR will release the resource occupied until its service completes. If a VNR is embedded with a low-quality solution, the network system will remain in a low resource utilization state until this VNR completes. As a result, more VNRs arriving later may be rejected due to scarce resources. Therefore, the uncertainty of upcoming VNRs presents a critical challenge in promoting the long-term benefits of Internet providers.

To address the above-mentioned challenges, we propose a **Hierarchical Reinforcement Learning (RL)** approach to learn a joint **Admission Control and Resource Allocation** policy for online VNE, named **HRL-ACRA**. Admission control and resource allocation are considered as upper-level and lower-level tasks in a hierarchical framework. We model both tasks as Markov decision processes (MDPs) and develop a hierarchical optimization framework based on the proximal policy optimization (PPO) [16] algorithm to train the corresponding policies. For each arriving VNR, an upper-level policy is responsible for deciding whether to admit it or not. The early rejection of inappropriate VNRs could reduce the unnecessary embedding process, thus improving resource utilization and accelerating the deployment speed. A lower-level policy is appointed to allocate resources of the physical network to meet the requirement of VNR. Specially, the average revenue method [17] is adopted to address the infinite horizon problem of the upper-level agent, and a multi-objective intrinsic reward is customized to alleviate the sparse reward issue of the lower-level agent. To fully use topological information and temporal relationship, we also design some deep neural network modules for sufficient feature extraction. Thanks to these carefully designed modules, HRL-ACRA achieves state-of-the-art (SOTA) performance with respect to several criteria.

To the end, our contributions are summarized as four-fold:

- We propose a hierarchical RL method for the admission control-aware online VNE problem. We cast admission control and resource allocation as upper-level and lower-level tasks in a hierarchical framework, model them as MDPs, and learn a joint policy. The upper-level agent takes the long-term benefits into consideration, while the lower-level agent pays close attention to the short-term profit.
- To address the infinite horizon problem caused by the continuous interactions between the upper-level agent and incoming VNRs, we adopt the average revenue method to

enhance the upper-level agent to treat the current and future revenue more equally. We also customize a multi-objective intrinsic reward with multiple local indicators to alleviate the sparse reward issue, encouraging the lower-level agent to perform exploration.

- We design a deep feature-aware graph neural network (GNN) with link feature awareness to make full use of topological features of the physical network and VNRs. Exploiting the topological information and temporal dependence simultaneously, we employ a sequence-to-sequence (Seq2seq) model based on this GNN and gated recurrent unit (GRU) to generate embedding actions iteratively.
- To simulate various real network situations, we conduct simulation experiments by adjusting the environment parameters. Extensive experimental results demonstrate that the proposed approach outperforms other SOTA algorithms on acceptance ratio and long-term average revenue.

The remainder of this paper is organized as follows. The related work is presented in Section II. In Section III, we provided the formulation of VNE problem and the basic concepts of GNN and RL. The details of our proposed algorithms are described in Section IV. In Section V, the experimental results and their analysis are given. Finally, we conclude this paper in Section VI.

II. RELATED WORK

Since the VNE problem is critical but challenging in network virtualization, studies for tackling it have received tremendous attention. In this section, we summarize the related works in two categories, the traditional and learning-based methods, and their brief information is concluded in Table I.

A. Traditional Methods

We categorize the traditional methods into three categories, the exact, heuristic, and meta-heuristic algorithms.

For the exact algorithms, the VNE problem can be typically formulated as mathematical programming. Chowdhury et al. [4] transformed VNE as a mixed-integer linear programming (MILP) by physical network augmentation and constraint relaxation, coordinating node, and link Mapping. Shahriar et al. [5] formulated VNE as an integer linear programming (ILP) problem, taking spare capacity location and request survivability into consideration. Diallo et al. [6] developed an ILP-based method for VNR splitting and a MILP-based approach for resource mapping to solve the multi-domain VNE problem. However, these algorithms cannot handle online deployment scenarios for their high computational complexity.

Instead of achieving optimal solutions, a wealth of heuristics-based algorithms are proposed to find feasible solutions quickly. Among them, node ranking is the primary strategy. Gong et al. [7] applied global resource capacity to evaluate the importance of nodes of the physical network and virtual network based on a random walk model, and then used the greedy matching strategy and the breadth-first search algorithm to conduct the node mapping and link mapping, respectively. A simple admission control strategy based on revenue-to-cost also was proposed to improve the long-term benefits. Still, it only

TABLE I
THE SUMMARY OF RELATED WORKS

Work	AC	Online	Method Category	Methodology	Feature Extraction	Exploited features
[4]	×	✓	exact	MILP	/	/
[5]	×	✓	exact	ILP	/	/
[18]	×	✓	exact	ILP and MILP	/	/
[7]	✓	✓	Heuristic	Node ranking	Global resource control	Physical network and virtual network
[8]	×	✓	Heuristic	Node ranking	Node resource management	Physical network and virtual network
[9]	×	✓	Heuristic	Priority of location	Evaluation on node and path	Physical network and virtual network
[19]	×	✓	Heuristic	Recursion and backtracking	/	/
[10]	×	✓	Meta-heuristic	Ant colony optimization	/	/
[11]	×	✓	Meta-heuristic	Particle swarm optimization	/	/
[12]	×	✓	Meta-heuristic	Overlapping decomposition	/	/
[20]	✓	✓	SL	Classification	RNN	Physical network and virtual network
[13]	×	✓	uSL	Subgraph exaction	Hopfield network	Only physical network
[21]	×	✓	uSL	Clustering	GAE	Only physical network
[14]	×	✓	RL	MCTS	/	Physical network and virtual network
[22]	×	✓	RL	REINFORCE	MLP	Physical network and current virtual node
[23]	×	✓	RL	Temporal-Difference	MLP	Physical network and virtual network
[24]	×	✓	RL	REINFORCE	CNN	Only physical network
[25]	×	✓	RL	REINFORCE	RNN	Only Physical Network
[15]	×	✓	RL	A3C	GCN	Physical network and current virtual node
[26]	✓	×	RL	Hierarchical RL with DQN	GCN	Physical network and current virtual node
Ours	✓	✓	RL	Hierarchical RL with PPO Average reward method	Seq2Seq Model Customized GNN	Physical Network, virtual network and VNR's global attributes

focused on the embedding solution of the current VNR without considering future VNRs. Similarly, Zhang et al. [8] adopted a capacity-based metric to sort nodes with multiple resource types.

Even though these algorithms are capable of generating feasible solutions quickly, they incur high blocking rates. Some schemes are also proposed to further improve the solution quality of VNE problems by handling failure situations. Yang et al. [19] studied the NP-hardness of the delay-sensitive and availability-aware VNE problem, and introduced a recursive heuristic method using the restricted shortest path algorithm method to solve it. Fan et al. [9] designed a priority of location VNE algorithm, following node proximity sensing and path comprehensive evaluation. Lin et al. [27] designed several heuristic algorithms for energy-aware VNE problem based on auxiliary graph building methods. Nevertheless, designing heuristics relies on rich expert knowledge and trial experience and only accommodates a few specific scenarios.

Moreover, many meta-heuristic algorithms have also been adopted to address the VNE problem and obtain near-optimal solutions. Fajjari et al. [10] presented an ant colony optimization algorithm for VNE problems that iteratively searches the solution spaces of node mapping. Su et al. [11] designed an energy-aware VNE algorithm based on the particle swarm optimization technique, regarding the node mapping solution as the position of particles. To solve large-scale VNE problems, Song et al. [12] proposed a divide-and-conquer evolutionary algorithm that utilizes the overlapping decomposition method to group the critical elements with tight connections to many other nodes into multiple sub-components. However, meta-heuristic algorithms are often non-deterministic and depend on extensive searches to discover high-quality solutions.

B. Learning-Based Methods

Recently, machine learning has proved to be a promising direction for solving combinatorial optimization problems.

According to the nature of the available learning signal, machine learning is usually categorized into three major categories: supervised learning (SL), unsupervised learning (uSL), and reinforcement learning (RL). Several works investigate machine learning applications to the VNE problem, and we introduce some of them following the three categories mentioned above.

Supervised learning has achieved great success in image recognition, text classification, and other scenarios. Treating the admission control mechanism of VNE as a binary classification problem, Andreas et al. [20] applied this idea to improve the existing VNE algorithm's performance by utilizing a Recurrent Neural Network (RNN) to judge whether to admit or reject arriving VNRs, but only considered the rejection of infeasible VNRs to avoid wasting time. Different from supervised learning relying on labeled samples, unsupervised learning is capable of discovering underlying structures in unlabeled data. To reduce the large search space, Blenk et al. [13] applied the Hopfield neural network to extract subgraphs fed to other existing VNE algorithms, resulting in faster and more resource-efficient embeddings. Habibi et al. [21] adopted the graph autoencoder (GAE) to cluster physical nodes and then randomly sample one node as the initial center to execute the breadth-first search (BFS) method in each cluster.

The embedding process of VNE can be regarded as a series of decisions, and some studies tackled this problem by reinforcement learning (RL). Modeling the node selection process as a Markov decision process (MDP), Haeri et al. [14] utilized the Monte Carlo tree search (MCTS) to improve the revenue-to-cost ratio. Wang et al. [23] trained a multi-layer perceptron (MLP) model with temporal-difference learning to approximate the value function of VNE states. Using a convolutional neural network (CNN) to extract the attributes of physical nodes and training it with REINFORCE algorithm, Li et al. [24] proposed a double-layer RL-based framework for VNE to learn the node mapping and link mapping process jointly. Yao et al. [25] designed a sequence-to-sequence model based on a recurrent

neural network (RNN) to exploit the historical information of previous decisions, and train with the REINFORCE algorithm. But these traditional neural networks often exhibit limited representational capabilities when dealing with non-euclidean data, requiring manual topological feature extraction like closeness and betweenness from graph theory. In contrast, graph neural networks (GNNs) excel at automatically mining deep features from graph data, generating richer representations [28]. Yan et al. [15] presented an RL-based method with a graph convolutional network (GCN) [29] to extract state features from the physical network, but did not explicitly exploit edge features. Cheng et al. [26] proposed a hierarchical RL control framework to select the VNR from batch candidates stored in the time window, where the GCN is used for the rough feature exploitation. However, their subagent for admission control only works for offline settings, e.g., within the time window batch, and is not unqualified to tackle the online VNE where future VNRs are not known in advance. In conclusion, existing RL-based works only concentrate on the resource allocation of arriving VNRs, or exploit partial latent temporal or topological features of the physical network and VNRs.

Compared with existing learning-based works, HRL-ACRA learns a joint admission control and resource allocation policy based on a hierarchical framework, capable of perceiving the long-term benefits and short-term interests. The well-designed neural network model also enables extracting sufficient temporal relationships and topological structures.

III. PRELIMINARIES

A. Admission Control-Aware Online VNE

1) *System Description*: Fig. 1 illustrates an example of the admission control-aware VNE Problem. In the practical network system, user services are virtualized as the virtual network requests (VNRs) set V , dynamically requesting resources provided by the physical network.

- *Physical Network* is defined as a weighted undirected graph $G^p = (N^p, L^p)$, where N^p and L^p respectively denote a set of physical nodes and a collection of physical links. The remaining node resources capacity of each physical node $n^p \in N^p$ is represented by C_{n^p} , while the remaining link resources capacity of each physical link $l^p \in L^p$ is characterized as B_{l^p} .
- *Virtual Network Request* is defined as a tuple $v = \langle G^v, A^v \rangle$, where G^v denotes a virtual network and A^v represents its global attributes. Here, A^v denotes the global attributes, e.g., lifetime. We describe the virtual network as a weighted undirected graph $G^v = (N^v, L^v)$, where N^v refers to a set of virtual nodes, and L^v is a collection of virtual links. Similarly, each virtual node $n^v \in N^v$ specifies node resources demand C_{n^v} , while each virtual link $l^v \in L^v$ indicates link resources demand B_{l^v} .

In standard online VNE workflow, the system will always attempt to allocate resources of physical network resources to each arriving VNR. The VNR will only be accepted if all virtual nodes and virtual links are mapped to physical nodes and physical paths, respectively, under the condition that the

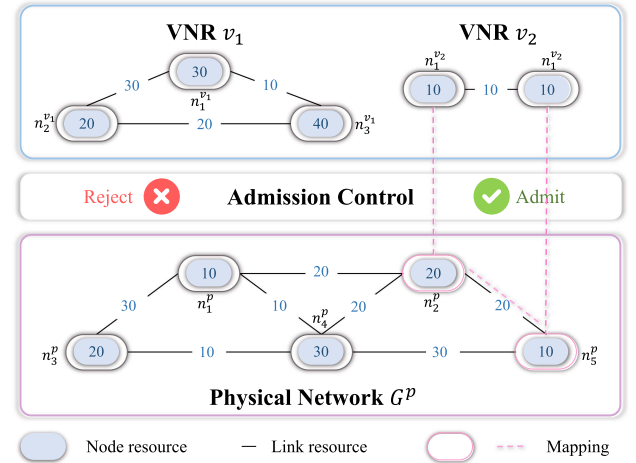


Fig. 1. Example of admission control-aware online VNE problem. For VNR v_1 , one of its virtual nodes, $n_4^{v_1}$, requires 40 units of node resources, while no physical node in the physical network G^p has enough available node resources to satisfy such demand. Due to constraint violations caused by insufficient resources, there is no feasible solution for v_1 . Therefore, admission control early rejected v_1 to skip the resource allocation process. In the case of VNR v_2 , all the resource demands of its virtual nodes and links can be accommodated by the physical network, satisfying all the constraints. As a result, v_2 can be admitted and successfully embedded. A feasible solution is shown with pink lines, where v_2 's nodes $n_1^{v_2}$ and $n_2^{v_2}$ are embedded into physical nodes n_2^p and n_5^p , respectively, and v_2 's link $(n_1^{v_2}, n_2^{v_2})$ is embedded into a physical path $[(n_2^p, n_5^p)]$. Hereafter, these mapped physical nodes (links) will update their available resources by subtracting the resource demand of corresponding virtual nodes (links).

constraints are satisfied. Once a VNR is successfully embedded, the resources will be occupied until the service completes. Since embedded VNRs cannot be discarded until their lifetime expires, each incoming VNR needs to compete for limited resources provided by the physical network with the currently serving VNRs. Therefore, the uncertainty of upcoming VNRs results in a critical challenge to promoting the long-term benefits of Internet providers.

Admission control (AC) is a proactive scheduling mechanism that has a potential impact on long-term benefits. It decides whether to admit or reject the incoming VNR, according to the VNR's requirements and the current situation of physical network. Considering the resource availability of the current physical network and the future VNR demand, the system can adaptively reject some VNRs to reserve resources for subsequent VNRs to improve long-term benefits, including acceptance ratio and long-term average revenue. For the unadmitted VNRs, they can further refine their orchestration [30] to enhance the probability of finding a high-quality feasible solution and request again. Additionally, early rejecting of VNRs in which no feasible solution exists and skipping the resource allocation phase contributes to speeding up the decision-making. Fig. 2 shows a comparison of without and with the admission control mechanism. Here, we emphasize that the status of admission control has long been ignored by most existing works. In this work, we try to utilize the hierarchical RL method to solve the VNE problem with more practical and challenging settings by jointly considering admission control and resource allocation.

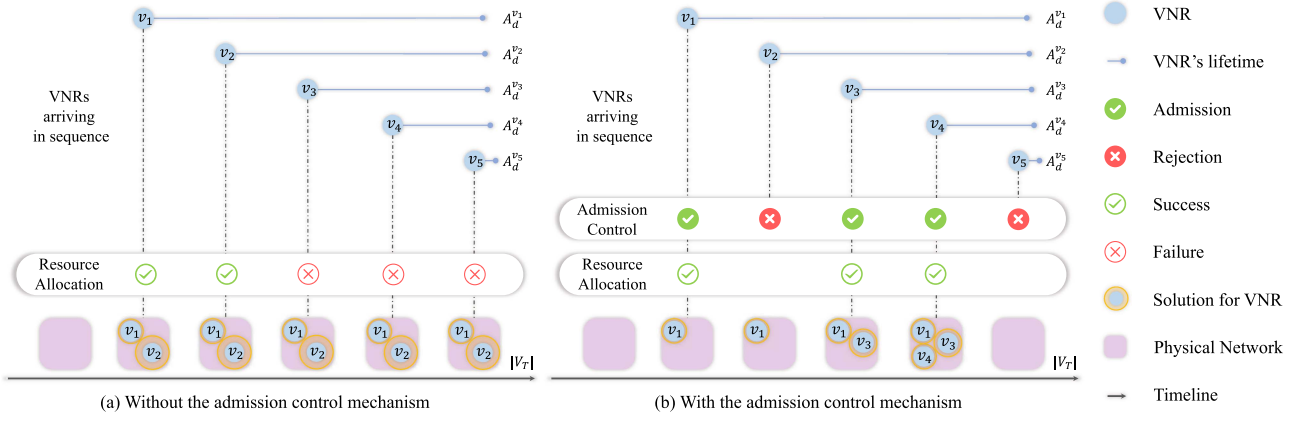


Fig. 2. Comparison of systems with and without admission control mechanism. (a) Without the admission control mechanism, all arriving VNRs will be attempted to execute resource allocation directly. After admitting and embedding VNR v_1 , this system embeds VNR v_2 with a low-quality solution and the majority of physical resources are conquered by v_1 , contributing to the rejection of v_3 , v_4 , and v_5 for the scarcity of physical resources. The resulting long-term average

and acceptance ratio of this system are $LA_Rev_a = \frac{\sum_{i \in \{1,2\}} (w_a + w_b * A_d^{v_i}) Rev(G^{v_i})}{|V^T|}$ and $AC_Ratio_a = \frac{2}{5}$, respectively. (b) With the admission control mechanism, dynamically arriving VNRs are selectively admitted and enter the resource allocation stage. Admission control proactively rejects VNR v_2 owing to the difficulty to find a high-quality solution, avoiding long-term low resource utilization. Then, sequentially arrived v_3 and v_4 are admitted and embedded successfully. For VNR v_5 , the system early rejects it due to the lack of available physical resources, skipping the unnecessary resource allocation process. The resulting long-term average and acceptance ratio of this system are $LA_Rev_b = \frac{\sum_{i \in \{1,3,4\}} (w_a + w_b * A_d^{v_i}) Rev(G^{v_i})}{|V^T|}$ and $AC_Ratio_b = \frac{3}{5}$, respectively. Using admission control makes the acceptance of v_3 and v_4 possible by actively rejecting v_2 , improving resource utilization. Compared with the absence of admission control mechanism, it achieves better performance on both the acceptance ratio and long-term average revenue. By early rejecting VNRs having no feasible solution, it also saves inference time and enhances the real-time decision.

The detailed description of the admission control-aware VNE procedure is summarized in Algorithm 1.

2) *Problem Formulation*: Embedding a VNR into the physical network can be defined as a mapping process from G^v to a subgraph of physical network $G^{p'}$ with various constraints.

$$f : G^v \rightarrow G^{p'}. \quad (1)$$

It is composed of node mapping and link mapping, where massive discrete variables need to be decided.

- *Node Mapping* is defined to assign all virtual nodes $n^v \in N^v$ to feasible physical nodes $n^p \in N^p$, constrained by

$$\sum_{n^p \in N^p} \phi_{n^p}^{n^v} = 1, \forall n^v \in N^v, \quad (2)$$

$$\sum_{n^v \in N^v} \phi_{n^p}^{n^v} \leq 1, \forall n^p \in N^p, \quad (3)$$

$$\phi_{n^p}^{n^v} C_{n^v} \leq C_{n^p}, \forall n^v \in N^v, \forall n^p \in N^p, \quad (4)$$

where $\phi_{n^p}^{n^v}$ is a binary variable that is set 1 when n^v is embedded in n^p . C_{n^v} and C_{n^p} denote node resource requirement and remaining capacity. Equation (2) and (3) are related to the relationship between every virtual node n^v and physical node n^p . Each virtual node $n^v \in G^v$ of the same virtual network must be placed on different network nodes. Equation (4) refers to the node resource constraint, i.e., the available resources of each physical node must exceed the resource request of the virtual node carried by it.

- *Link Mapping* is defined to embed each virtual links $l^v \in L^v$ into one loop-free physical path $p_{l^v}^p \in P^p$, constrained

by

$$B_{l^v} \leq B_{l^p}, \forall l^p \in p_{l^v}^p, \quad (5)$$

where P^p is defined as a set of all loop-free physical paths in the physical network. $p_{l^v}^p$ is one element of P^p that connects two physical nodes that accommodate the source and destination nodes of the virtual link l^v . B_{l^v} and B_{l^p} denote the bandwidth resource requirement and remaining capacity. Equation (5) means the available bandwidth of each physical link in the physical path must exceed the bandwidth request of the virtual link. The pink lines in Fig. 1 depict an example of the mapping process of one virtual link.

If node mapping or link mapping fails due to constraint violations, it indicates that a feasible solution for the VNR has not been found, which means the VNR cannot be embedded.

3) *Performance Evaluation*: The most important performance evaluation metrics are acceptance ratio (AC_Ratio), long-term average revenue (LA_Rev), and running time [31], which are defined as:

- *Acceptance Ratio* measures the QoS with the number of the accepted VNRs, which is defined as

$$AC_Ratio = \lim_{T \rightarrow \infty} \frac{\sum_{v \in V^T} \mathbb{I}(v)}{|V^T|}, \quad (6)$$

where $\mathbb{I}(v)$ is the indicator function that returns 1 if the VNR v is accepted and 0 otherwise.

Algorithm 1: Admission Control-Aware VNE Procedure.

Input : A VNE solver S ; The arriving VNR
 $v = \langle G^v, A^v \rangle$, where $G^v = (N^v, L^v)$;
The current physical network $G^p = (N^p, L^p)$;

Output: The embedding result for the VNR v ;

```

1  /**Admission Control**/;
2  if  $v$  is Admitted then
3    |  $S$  executes the Resource Allocation process;
4  else
5    | return False;
6  end
7  /**Resource Allocation**/;
8  for Virtual node  $n_i^v \in N^v, i = 1, 2, \dots$  do
9    |  $S$  finds a physical node  $n_i^p$  for virtual node  $n_i^v$ ;
10   |  $S$  attempts to place  $n_i^v$  onto  $n_i^p$ ;
11   if  $n_i^v$  is successfully placed then
12     | for Virtual node  $n_j^v \in N^v, i = 1, 2, \dots$  do
13       | if  $(n_i^v, n_j^v) \notin L^v$  then
14         | continue;
15       end
16       |  $S$  finds a physical path  $p_{i,j}^p$  for  $(n_i^v, n_j^v)$ ;
17       |  $S$  attempts to route  $(n_i^v, n_j^v)$  onto  $p_{i,j}^p$ ;
18       if  $(n_i^v, n_j^v)$  is successfully routed then
19         | continue;
20       else
21         |  $S$  undoes all the previous actions;
22         | return False;
23       end
24     end
25   else
26     |  $S$  undoes all the previous actions;
27     | return False;
28   end
29 end
30 return True;

```

- *Long-Term Average Revenue* is a direct indicator to reflect the revenue of Internet providers, which is defined as

$$LA_Rev = \lim_{T \rightarrow \infty} \frac{\sum_{v \in V_T} \mathbb{I}(v)(w_a + w_b * A_d^v) \text{Rev}(G^v)}{|V_T|}, \quad (7)$$

where $V_T = \{v | 0 < t_v < T\}$ is the set of VNRs arriving before time instance T . A_d^v donates the lifetime of VNR v . w_a and w_b are the weight of starting price and the weight of the service time charge, respectively. Internet providers can make various pricing strategies by deciding the value of w_a and w_b to control the two components of price. $\text{Rev}(G^v)$ is the basic revenue of a VNR calculated by the total amount of resource requests of its nodes and links, computed by

$$\text{Rev}(G^v) = \sum_{n^v \in N^v} C_{n^v} + \sum_{l^v \in L^v} B_{l^v}. \quad (8)$$

- *Running Time* describes the time consumption of the VNE algorithm to process VNRs. In practice, a VNE algorithm needs to face online and dynamic scenarios, which requires a high real-time guarantee. Consequently, it is necessary

that the used VNE algorithm should balance the running time and performance, arranging the arriving VNR as soon as possible to meet the real-time requirements.

In this work, we jointly optimize the long-term average revenue and acceptance ratio, while considering running time.

B. Graph Neural Network

Graph neural networks (GNN) that can operate on non-European data have become a hot topic in the field of deep learning recently, which enables deep learning technology to be competent for more complex tasks. Based on the message propagation mechanism between nodes, GNN models the relationships and dependencies between nodes to extract the deep-level information in the graph-structured data [28]. A variety of GNNs can be divided into several categories, one of which is the spatial-based graph convolutional neural network. By designing an aggregation function and adopting a message propagation mechanism, the spatial-based graph convolutional neural network updates the central node's representation by aggregating its neighbor nodes' representation.

Graph attention network (GAT) [32] is a typically spatial-based graph convolutional neural that integrates the self-attention mechanism into the propagation step. To obtain the new representation of one node, GAT computes adaptive attention coefficients between two nodes to aggregate representations of neighbor nodes. Formally, the graph convolutional operation of GAT ℓ th layer is defined by

$$h_i^{(\ell)} = \sigma \left(\sum_{j \in N(i) \cup \{i\}} \alpha_{i,j} W h_j^{(\ell-1)} \right), \quad (9)$$

where $h_i^{(\ell)}$ and $h_i^{(\ell-1)}$ are the node representation of node i in the ℓ th and $\ell - 1$ th GNN layer, respectively. σ is an activation function and W is a learnable weight to linear project the node input node representation. The coefficient $\alpha_{i,j}$ between node i and node j is computed as

$$\alpha_{i,j} = \frac{\exp \left(\sigma \left(W_a \left[h_i^{(\ell)} \| h_j^{(\ell)} \right] \right) \right)}{\sum_{k \in N(i) \cup \{i\}} \exp \left(\sigma \left(W_a \left[h_i^{(\ell)} \| h_k^{(\ell)} \right] \right) \right)}. \quad (10)$$

Here, $\|$ is the concatenation operation. $N(i)$ is the set of node i 's neighbor nodes and W_a is a trainable attention weight vector.

Graph pooling networks are usually utilized to obtain the global representation of one whole graph for conducting graph-level tasks. Graph attention pooling (GAP) is one of the powerful graph pooling networks that considers both node features and graph topology based on self-attention [33]. GAP aggregates all node representations h by doing a weighted summation, where a context is employed to compute the attention coefficients. The context is defined as the linear transformation of all node representations' mean pooling

$$c = \sigma \left(\left(\frac{1}{|N|} \sum_{i=1}^{|N|} h_i \right) W \right), \quad (11)$$

where $|N|$ denotes the number of nodes and W is a learnable weight matrix. Using this context c , the attention coefficient of node i is given by

$$a_i = h_i^{Tr} c. \quad (12)$$

Here, Tr represents the transpose operation. Finally, the graph-level representation is computed as the weighted sum of all node representations

$$g = \sum_i^{|N|} a_i h_i. \quad (13)$$

C. Reinforcement Learning

Reinforcement Learning (RL) is an experience-driven learning framework widely used to solve sequential decision-making problems, where an agent continuously interacts with an environment to collect experiences for learning. In RL, Markov decision process (MDP) serves as a general mathematical model. It can be defined as a tuple $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$, where $\mathcal{S}$ denotes the state space of environment, $\mathcal{A}$ represents the action space of agent. $P: \mathcal{S} \times \mathcal{A} \rightarrow \mathcal{S}$ indicates the transition function from the current state to the next state in response to the selected action. $R: \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow \mathbb{R}$ is the immediate reward function to evaluate the decision quality at the current state, mapping from states and actions into real numbers. $\gamma \in [0, 1]$ is a scalar discount factor to regulate the agent's attention to short-term rewards and long-term returns. Specifically, the agent selects an action a_t based on the observed state s_t from the environment at each decision timestep t . After the agent executes the action a_t , the environment will transition to a new state s_{t+1} and feedback a reward r_t to the agent. The objective of the agent is to find an optimal policy π^* , a mapping function from states into actions, to maximize the expected return

$$\pi^* = \operatorname{argmax}_{\pi} \mathbb{E}[G_t], \quad (14)$$

where $G_t = \sum_t^t \gamma^t r_t$ denotes cumulative discounted reward.

Policy gradient is a kind of mainstream reinforcement learning algorithm, which computes an estimator of the policy gradient for gradient ascent algorithm to optimize the policy. The optimization objective of the most classic gradient estimator is

$$L(\theta) = \hat{\mathbb{E}}_t [\log \pi_{\theta}(a_t | s_t) \hat{A}_t], \quad (15)$$

where π_{θ} is a policy parameterized by θ and $\hat{A}_t$ is an advantage estimator at timestep t . The simplest advantage estimator uses the average return over several episodes as the baseline. More practically, a learnable value function can assist in the advantage estimator, which estimates the expected return of a given state

$$V_{\pi_{\theta}}(s_t) = \hat{\mathbb{E}}_t [G_t | s_t]. \quad (16)$$

These approaches in such form are unified under the actor-critic architecture where the actor (policy) learns a policy to mask decisions and the critic (value function) estimates the expected return of the state. Benefit from the critic working as a baseline, the advantage estimator can be computed by

$$\hat{A}_t = r_t + V_{\pi_{\theta}}(s_{t+1}) - V_{\pi_{\theta}}(s_t). \quad (17)$$

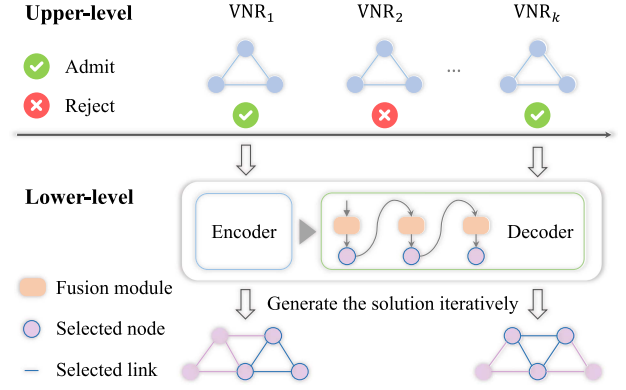


Fig. 3. Overall framework of our proposed hierarchical method for admission control-aware VNE.

By doing this, the bias estimated by the value function is introduced to trade-off variance reduction policy gradients.

IV. METHODOLOGY

To solve this challenging combinatorial optimization problem of VNE, we propose a **Hierarchical Reinforcement Learning** based approach to learn a joint **Admission Control and Resource Allocation** policy of VNE, named **HRL-ACRA**. As illustrated in Fig. 3, we cast the admission control and resource allocation as upper-level and lower-level tasks, respectively. The upper-level agent has a foresight capability, whose goal is to optimize the long-term benefits, including the acceptance ratio and long-term revenue. According to the current situation of physical network and the request information of incoming VNR, it decides whether to admit the arriving VNR. The lower-level agent focuses on the resource allocation for admitted VNRs to generate high-quality solutions. To reduce the action space, we utilize a sequence-to-sequence (seq2seq) model to construct the solution iteratively rather than in one step, i.e., the embedding actions of each virtual node are generated sequentially. Considering the state of VNRs is consistent, this seq2seq model consists of a static encoder and a dynamic decoder. The encoder extracts the decision order and features of nodes and links as node representations. The decoder selects a physical node to accommodate the current virtual node iteratively, based on the information aggregated by a fusion module at each timestep. Two agents are trained by the proximal policy optimization (PPO) method.

A. Upper-Level Agent

The upper-level agent is responsible for deciding on admission or rejection of arriving VNRs, which optimizes the acceptance ratio and the long-term revenue of Internet providers.

1) *MDP Definition*: We model whether to admit the arriving VNR as an MDP, which contains three key elements: state, action, and reward.

- *State*: The upper-level state consists of the request information of incoming VNR and the current situation of the physical network, shown in Table II. Specifically, the

TABLE II
THE UPPER-LEVEL AND LOWER-LEVEL STATES

	The Upper-level State	The Lower-level State
Physical Network	Remaining resources of each physical node	Remaining resources of each physical node
	Remaining resources of each physical link	Remaining resources of each physical link
	Indexes of physical links	Indexes of physical links
		Maximum resources of each physical node
Virtual Network	Resource demands of each virtual node	Resource demands of each virtual node
	Resource demands of each virtual link	Resource demands of each virtual link
	Indexes of virtual links	Indexes of virtual links
		Neighbor flag of physical nodes
VNR's Global Attributes	Lifetime of the VNR	Remaining number of virtual nodes to be embedded

request information of incoming VNR includes node resource demands and link resource requirements, topology structure, and lifetime. And the current situation of the physical network is composed of maximum and available resources of all nodes and links, and topology structure.

- *Action*: The upper-level action $\hat{a} \in \hat{\mathcal{A}} = \{0, 1\}$ is a binary variable that indicates whether to admit the incoming VNR.
- *Reward*: The extrinsic reward for the upper-level agent is designed to guide to optimize the long-term average revenue, also considering the revenue-to-cost ratio and avoidance of failed embedding to enable assessment of more situations, defined as

$$\hat{r} = \begin{cases} \frac{\text{Rev}(G^v)}{\text{Cost}(G^v)} \text{Rev}(G^v), & \text{if } G^v \text{ is admitted and embedded,} \\ -w_1, & \text{if } G^v \text{ is admitted but not embedded,} \\ 0, & \text{otherwise.} \end{cases} \quad (18)$$

Here, $\frac{\text{Rev}(G^v)}{\text{Cost}(G^v)}$ denotes the revenue-to-cost ratio. Similar to the $\text{Rev}(G^v)$, $\text{Cost}(G^v) = \sum_{n^v \in N^v} C_{n^v} + \sum_{l_v \in L_v} \sum_{l_p \in p_{l_v}^p} B_{l_v}$ is computed by the total amount of resource consumption for embedding G^v . A higher revenue-to-cost ratio means fewer resources are consumed to embed the current VNR, saving more resources for future VNRs. When an arriving VNR is admitted and then successfully embedded, we return a positive reward to encourage the agent to satisfy as many requests as possible. Considering different embedding solutions for the same VNR and the physical network, a better revenue-to-cost ratio can contribute to greater reward. For admitted but unsuccessfully embedded VNRs, we feedback a small negative reward $-w_1$ to prevent the agent from admitting infeasible VNRs, which contributes to omitting the resource allocation phase and improving the decision speed.

2) *Policy Network*: Since both the physical network and VNRs are graph-structured, GNN is a suitable and powerful feature extractor that is employed in our framework. GCN [29] and GAT [32] are the two best representative GNN models. GAT allows for adaptive weights for each neighbor node when aggregating the node features, while GCN assigns the fixed

weight to all neighbor nodes. This attention mechanism enables GAT to adaptively model the importance of different neighborhood nodes and capture more fine-grained information about the neighborhood. Therefore, GAT is more powerful and flexible than GCN, especially in our tasks where fine-grained information about the graph structure is important.

Unlike traditional graph tasks, link features are significant for VNE to perceive bandwidth resources. To engage bandwidth awareness with our deep model, we extend the GAT to blend link features into nodes during the propagation process. Besides, the shortest paths determined during the link mapping stage may consist of multiple hops. As such, it is necessary to build a deep GNN to learn a comprehensive node representation by leveraging information from multi-hop neighbors. However, the use of many layers in the construction of a GNN can lead to the problem of over-smoothing. To mitigate this issue, we deepen the GNN by using initial residual connections [34] and identity mapping [35].

GAT is capable of extracting node features and topology information from non-European data. Still, it is not directly applicable to the VNE problem where link features play an important role. Consequently, we extend GAT to blend link features into node embedding, bringing awareness of bandwidth resources. Based on (10), introducing link features into the calculation of coefficients, the attention coefficient between node i and j is computed by

$$\alpha_{i,j} = \frac{\exp\left(\sigma\left(W_a\left[h_i^{(\ell)} \parallel h_j^{(\ell)} \parallel h_{i,j}\right]\right)\right)}{\sum_{k \in N(i) \cup \{i\}} \exp\left(\sigma\left(W_a\left[h_i^{(\ell)} \parallel h_k^{(\ell)} \parallel h_{i,k}\right]\right)\right)}.$$

where $h_{i,j} = \text{MLP}(B_{e_{i,j}})$ is the edge representation of link $e_{i,j}$, obtained by pass its link features $B_{e_{i,j}}$ into an MLP.

Moreover, deepening the number of GNN layers has been shown to be more effective in mining information from the graph, but it also confronts the challenge of over-smoothing. Therefore, we customize an attribute-aware GNN using the initial residual connection and identity mapping to alleviate this problem. Formally, the operation of this GNN ℓ th layer is defined by

$$h_i^{(\ell)} = \sigma\left[\sum_{j \in N(i) \cup \{i\}} ((1 - \alpha)\alpha_{i,j}h_j^{(\ell-1)} + \alpha h_j^{(0)})\right] \cdot ((1 - \beta)I_n + \beta W). \quad (19)$$

Here, α and β are two hyperparameters denoting the strength of the initial residual connection and the identity mapping, respectively. h_j^0 is the initial node representation of node j . An identity mapping $I_n = f(H^{(\ell)})$ is added to the ℓ th weight matrix $W^{(\ell)}$.

The direct use of node representations is computation-intensive, we adopt graph attention pooling (GAP) to obtain the graph-level representation based on the attention mechanism [36]. Specifically, when a VNR v arrives in the system, the physical network G^p and the virtual network G^v of this VNR are encoded into node representations as $\hat{h}^p$ and $\hat{h}^v$ respectively by this GNN

$$\hat{h}^p = \text{GNN}(G^p), \hat{h}^v = \text{GNN}(G^v). \quad (20)$$

We then adopt graph attention pooling (GAP) to obtain the graph-level representations of physical and virtual network. Meanwhile, VNR's global attributes A^v are perceivable by MLP

$$\hat{g}^p = \text{GAP}(\hat{h}^p), \hat{g}^v = \text{GAP}(\hat{h}^v), \hat{g}^r = \text{MLP}(A^v). \quad (21)$$

We fuse them with an MLP and then pass the result and last hidden state $\hat{h}$ to a GRU to receive the current hidden state $\hat{h}'$

$$\hat{h}' = \text{GRU}(\text{MLP}(\hat{g}^p, \hat{g}^v, \hat{g}^r), \hat{h}). \quad (22)$$

The final upper-level action probabilities $\hat{\pi}$ is generated by

$$\hat{\pi} = \text{Softmax}(\text{MLP}(\hat{h}')). \quad (23)$$

B. Lower-Level Agent

1) *MDP Definition*: When a VNR is admitted by the upper-level agent, the lower-level agent will attempt to find an embedding solution to allocate resources for it. Similarly, we model the resource allocation process of VNE as an MDP, which is defined as follows.

- *State*: Compared to the upper-level state, the lower-level state needs more detailed information on the resource allocation situation for the same VNR. This is due to the state changes that occur after a virtual node is successfully embedded. Specifically, the lower-level state is composed of the current situation of the physical network and virtual network, and VNR's global attributes, which are similar to the upper-level state. Notably, we design two binary flag features to reflect the status of physical nodes at each timestep, including (1) Selection flag of physical nodes indicates whether the physical nodes were selected. The flag value is 1 if the physical node was already selected; otherwise, 0; (2) Neighbor flag of physical nodes represents whether the neighbor of the current virtual node is placed in the physical node. If true, the value is 1; otherwise, 0. The detail of the lower-level state is shown in Table II.
- *Action*: The lower-level action space $\mathcal{A}$ is a subset of physical nodes, where we utilize a mask vector to dynamically remove nodes with insufficient resources or that have been selected before. An action $a \in \mathcal{A}$ is selected to accommodate the current virtual node. Then, we utilize the available graph construction-based shortest path method [7] to execute the link mapping.

- *Reward*: There is no guiding signal for the lower-level agent until the VNR is completely embedded or rejected, resulting in a sparse reward issue. To alleviate this issue, we develop a multi-objective intrinsic reward with multiple local indicators to guide the agent to perform efficient exploration, which is defined as

$$r = \begin{cases} \delta + \frac{\text{Rev}(G^v)}{\text{Cost}(G^v)}, & \text{if } G^v \text{ is embedded successfully,} \\ \delta, & \text{if } a_t \text{ is successful,} \\ -\frac{1}{|N^v|}, & \text{otherwise,} \end{cases} \quad (24)$$

where $\delta = \frac{1}{|N^v|}(\frac{\text{Rev}_t(G^v)}{\text{Cost}_t(G^v)} + w_2\psi(a_t))$ is a temporal signal to estimate the quality of current partial solution at each timestep t , considering both newly revenue-to-cost ratio and load balancing. $\frac{\text{Rev}_t(G^v)}{\text{Cost}_t(G^v)}$ is the newly revenue-to-cost ratio computed by the revenue divided by the cost at timestep t . $\psi(a_t)$ is defined as the resource load balancing of physical nodes a_t , calculated by the ratio of the remaining resource to the maximum resource of a_t . w_2 denotes the weight of node load balancing. A high revenue-to-cost ratio and resource load balancing can reserve more resources for subsequent VNRs. Once the G^v is accepted, i.e., $t = |N^v|$ and a_t is successful, we additionally return the complete revenue-to-cost ratio to award the agent for finding a feasible embedding solution.

2) *Policy Network*: As shown in Fig. 4, a seq2seq model, containing a static encoder and a dynamic decoder, is employed to extract the static and dynamic features respectively to seek a trade-off between running time and performance.

- *Encoder*: For the fixed state of VNR, we adopt a static GNN encoder to learn the feature embedding of each virtual node. Additionally, the decision order of each virtual node is usually preset, which is generally neglected by existing learning-based methods and incapable of being handled by GNN. Therefore, a position encoder [37] is also used to generate the position embedding for each virtual node h_p^v , which is concatenated (CONCAT) with the virtual node's GNN embedding h_f^v to form the VNR's node embedding h^v as follow:

$$h_f^v = \text{GNN}(g^v), h_p^v = \text{PE}(g^v), \quad (25)$$

$$h^v = \text{CONCAT}(h_f^v, h_p^v). \quad (26)$$

- *Decoder*: At each timestep t , we encode the state of the physical network G_t^p with GNN to extract node embeddings and use a GAP layer to conglomerate them to obtain the graph embedding. Simultaneously, the global attributes of VNR are sent into an MLP as follows:

$$g_t^p = \text{GAP}(\text{GNN}(G_t^p)), g^r = \text{MLP}(A^v). \quad (27)$$

The fusion embedding at timestep t is renewed by integrating the current virtual node embedding h_t^v , the VNR's global attributes representation, and the physical network's graph embedding. And then, the fusion embedding is input into a GRU, pulling in the last hidden state h_{t-1}

$$h_t = \text{GRU}(\text{MLP}(g_t^p, h_t^v, g^r), h_{t-1}). \quad (28)$$

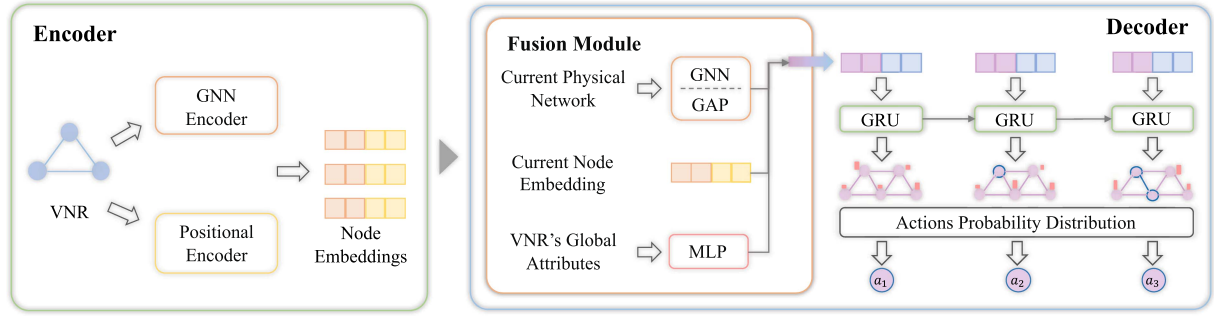


Fig. 4. Overview of the policy network of the lower-level agent. i) Encoder: The embeddings of each virtual node are composed of the feature embedding extracted by GNN Encoder and position embedding generated by the positional encoder. ii) Decoder: At each timestep t , the GRU-based decoder iteratively generates the embedded actions for each virtual node by aggregating the current embedding of the virtual node, the situation of physical network, and VNR's global attributes with the fusion module.

Finally, we obtain the lower-level action probability distribution

$$\pi_t = \text{Softmax}(\text{MLP}(h_t)). \quad (29)$$

C. Training Method

To enhance the stability and efficiency of training, the proximal policy optimization (PPO) [16] is adopted to train two agents. PPO is an improved policy gradient algorithm based on actor-critic architecture, whose objective function is following:

$$L(\theta) = \min \left(r_\theta \cdot \hat{A}, \text{clip}(r_\theta, 1 - \epsilon, 1 + \epsilon) \cdot \hat{A} \right), \quad (30)$$

where $r_\theta = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$ denotes the probability ratio defined by the ratio of the current policy with parameter θ to the pre-update policy with parameter θ_{old} , $\hat{A}$ represent an estimator of the advantage function computed by the discounted accumulative reward minus the prediction value of critic network, and ϵ indicates a hyperparameter clipping the probability ratio.

The upper-level agent continuously tackles arriving VNRs in the practical scenario, whose interaction with the environment can be modeled as an infinite-horizon MDP. The RL methods using discounted rewards are difficult to adapt to the infinite horizon MDP because they cannot treat the current and future revenue equally. Therefore, we utilize the average reward method [17] to modify the advantage estimator (17) to allow the upper-level agent to learn policies that maximize average long-term returns

$$\hat{A}_t = r_t - \rho + V_{\pi_\theta}(s_{t+1}) - V_{\pi_\theta}(s_t), \quad (31)$$

where $\rho = \frac{1}{T} \sum_{t=1}^T r_t$ is the average reward for T timesteps.

Initially, we train the lower-level agent to learn a great resource allocation policy in the default experimental setting, where this agent tackles a diverse range of VNRs and endows its policy with excellent generalizability. As a result, this well-trained policy consistently produces high-quality solutions across various scenarios [15], and we freeze it in the subsequent process. Then, our focus then shifts towards training the upper-level agent, to learn adaptive admission control strategies tailored to different experimental settings. This process ensures both training efficiency and adaptability of HRL-ACRA.

Algorithm 2: The Training Process of Lower-Level Agent.

Input : The initial parameters of the lower policy θ ;

Output: The optimized parameters of the lower policy θ ;

```

1 for iteration = 1, 2, ... do
2   for  $v_i, i = 1, 2, \dots$  do
3     Get the initial state  $s_0$  from environment;
4      $s_t \leftarrow s_0$ ;
5     Encode the state of VNR with encoder;
6     for Virtual node  $n_j^v \in N^v, j = 1, 2, \dots$  do
7       Generates the action probability distribution
8          $\pi_t$  with decoder;
9       Sample an action  $a_t$  based on  $\pi_t$ ;
10      Attempt to place  $n_j^v$  onto  $a_t$ ;
11      if  $n_j^v$  is successfully placed then
12        Execute the link mapping;
13        if Link mapping is unsuccessful then
14          Undo all the previous actions;
15        end
16      else
17        Undo all the previous actions;
18      end
19      Receive the intrinsic reward  $r_t$ ;
20      Get the next state  $s_{t+1}$ ;
21      Collect experience  $\{s_t, a_t, r_t, s_{t+1}\}$ ;
22      if Update then
23        Optimize the parameters of policy  $\theta$ ;
24      end
25      if Node placing or link mapping fails then
26        break
27      end
28       $s_t \leftarrow s_{t+1}$ ;
29    end
30  end

```

We also apply the advantage normalization and entropy regularization [38] methods to improve the efficiency of training exploration. In the training phase, two agents sample actions according to generated policies. The detailed training scheme of two agents is summarized in Algorithms 2 and 3. Specially, both agents interact with their environment to gather experience.

Algorithm 3: The Training Process of Upper-Level Agent.

Input : The initial parameters of the upper policy $\hat{\theta}$;
Output: The optimized parameters of the upper policy $\hat{\theta}$;

```

1 for iteration = 1, 2, ... do
2   Get the upper-level state  $\hat{s}_0$ ;
3    $\hat{s} \leftarrow \hat{s}_0$ ;
4   for  $v_i, i = 1, 2, \dots$  do
5     Upper-level agent generates action probability
       distribution  $\hat{\pi}$ ;
6     Upper-level agent samples action  $\hat{a}$  based on  $\hat{\pi}$ ;
7     if  $\hat{a} = \text{admit}$  then
8       Lower-level agent attempt to allocate
         resources for VNR $_i$ .
9     end
10    Upper-level agent receives the extrinsic reward  $\hat{r}$ ;
11    Upper-level agent gets the next state  $\hat{s}'$ ;
12    Upper-level agent collects experience  $\{\hat{s}, \hat{a}, \hat{r}, \hat{s}'\}$ ;
13     $\hat{s} \leftarrow \hat{s}'$ ;
14    if Update then
15      Update the parameters of upper-level policy  $\hat{\theta}$ ;
16    end
17  end
18 end

```

Algorithm 4: Inference With HRL-ACRA.

Input : A series of VNRs V arriving at the physical network sequentially;
Output: Embedding results of each VNR Φ ;

```

1 for  $v_i, i = 1, 2, \dots$  do
2   Upper-level agent decides whether to admit it;
3   if admitted then
4     Lower-level agent generates multiple solutions
       with beam search;
5     if exist feasible solutions then
6       Select the best solution to allocate resources;
7        $\Phi \leftarrow \Phi \cup \{\text{True}\}$ ;
8     else
9        $\Phi \leftarrow \Phi \cup \{\text{False}\}$ ;
10    end
11  else
12     $\Phi \leftarrow \Phi \cup \{\text{False}\}$ ;
13  end
14 end

```

Each agent generates an action probability distribution based on current state and samples an action following this distribution. The environment transits to new states and feedbacks rewards to the agent. At intervals of specified timesteps, they utilize this accumulated experience to optimize their parameters. Algorithm 4 describes the inference workflow of HRL-ACRA for tackling a series of sequentially arriving VNRs. For each incoming VNR, the upper-level agent first decides whether to admit it. If admitted, the lower-level agent will parallelly generate multiple solutions with the beam search strategy [39] and select the best feasible solution to embed it, which contributes more sufficient exploration of solution space.

TABLE III
PARAMETER SETTINGS

Group	Parameter	Value
Neural Network	Number of GRU layers	1
	Hidden dimension of GRU	128
	Number of MLP layers	3
	Number of GNN layers	5
	Embedding dimension of GNN	128
	Strength of the initial residual connection	0.2
Model Training	Strength of the identity mapping	0.2
	Learning rate of actor	0.0010
	Learning rate of critic	0.0005
	Batch size	256
Reward Weights	Discount factor of reward γ	0.99
	Weight of negative reward w_1	0.1
	Weight of node load balancing w_2	0.01

V. EXPERIMENTS

In this section, we conduct extensive experiments to demonstrate the effectiveness of HRL-ACRA algorithms, by summarizing the main implementation details and result analysis. Due to the limitation of page length, we place the baseline descriptions, ablation studies, and hyperparameter sensitivity analysis in Appendix, available online.

A. Experimental Setup

1) *Implementation Details:* Following the settings of most previous works [7], [15], [22], we implement a typical VNE simulator to evaluate the performance of multiple algorithms under the same platform. The topology of the physical network is generated following a Waxman random graph with the parameters $\alpha = 0.5$ and $\beta = 0.2$, representing a medium-sized network system with 100 nodes and about 500 links. The central processing unit (CPU) and bandwidth are considered as node and link resources, respectively. The node and link resources of physical network satisfy the uniform distribution from 50 to 100 units. In one simulation of the training and testing phase, we randomly generate 1,000 VNRs to arrive at the system successively following a Poisson process with an average arrival rate of 4 per 100 time units. Each VNR has an exponentially distributed lifetime with an average of 1,000 time units. By default, every VNR has [2–10] nodes following the uniform distribution, and each pair has a half probability of being connected to form a virtual link. Besides, the demands of VNR for node and link resources are uniformly distributed from 0 to 50.

All learning-based models are implemented with PyTorch. The trainable weights are initialized with the Xavier normal method and updated with the Adam optimizer [40]. Specifically, in the testing phase, the upper-level agent greedily chooses actions with the highest probability, while the lower-level agent develops multiple solutions using the beam search strategy. Then, the physical network will allocate resources utilizing the feasible solution with the least cost. Other parameter settings are divided into three groups, neural network, model training, and intrinsic reward, shown in Table III. The hardware we utilize to train models is a server of Ubuntu 18.04.2 LTS with 48 Intel(R) Xeon(R) Silver 4214 CPU @ 2.20 GHz and 1 Nvidia GeForce RTX 3090 GPU.

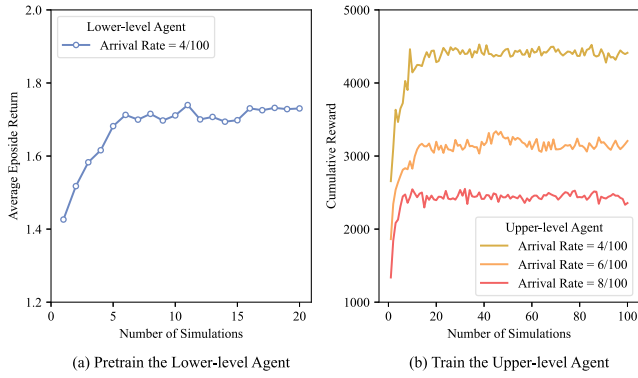


Fig. 5. Learning curves of the lower-level and upper-level agent.

2) *Compared Baselines*: To validate the effectiveness of the proposed HRL-ACRA, we use the LA_Rev and AC_Ratio defined in Section III as main evaluation indicators and compare it with six existing algorithms, which cover three classical heuristic-based algorithms, including *GRC* [7], *NRM* [8] and *PL* [9], and four learning-based algorithms, including *MCTS* [14], *A3C-GCN* [15], *REINFORCE-CNN* [24] and *GAE-BFS* [21]. The hyper-parameters of baseline algorithms are set to be the same as those mentioned in their original papers.

B. Result Analysis

The realistic network environment is usually complex and changeable. We simulate various network environments by adjusting some environment settings to verify the performance of these algorithms in dealing with real scenarios.

1) *Convergence Analysis*: The learning curves on pretraining the lower-level agent and jointly training two agents in different settings of arrival rate are illustrated in Fig. 5. In each simulation, the lower-level agent focusing on VNR resource allocation and the upper-level agent having an infinite horizon, experience 1,000 episodes and 1,000 timesteps, respectively, to optimize their policies. We can observe that the lower-level agent achieves fast convergence at about the sixth episode in the pretraining process. Due to the infinite horizon of the upper-level agent, we opt to assess the learning progress by utilizing the cumulative reward of one simulation, namely, the total reward accumulated over 1,000 timesteps. In the training process of three arrival rate settings, the upper-level agent also achieves fast convergences. HRL-ACRA can converge after an acceptable amount of training, which manifests our reward design's effectiveness in guiding policy optimization.

2) *Arrival Rate Tests*: In different periods, network systems often confront various service request traffic. To evaluate the ability of VNE algorithms to handle various frequencies of VNRs, we run these algorithms in three frequent arrival scenarios by setting the arrival rate to 0.04, 0.06, and 0.08, respectively. The results are presented in Table V.

We can see that as the arrival rate of VNRs increases, acceptance rates of all algorithms decrease because more VNRs compete for the resources of the physical network in the same period. When the arrival rate is set to 0.08, our proposed HRL-ACRA

TABLE IV
AVERAGE RUNNING TIME* IN NODE SIZE TESTS

Method	Node Size		
	2-10	2-15	2-20
GRC	128.01	141.56	155.35
NRM	95.54	100.83	106.33
PL	190.26	238.06	352.61
MCTS	953.38	2047.33	3244.17
A3C-GCN	169.91	184.03	285.41
REINFORCE-CNN	133.41	180.10	233.84
GAE-BFS	98.41	140.29	175.18
HRL-ACRA (Ours)	161.27	173.82	205.37

*Average of 10 experiments results (in seconds)

increases the AC_Ratio by about 18.93%, 10.31%, and 5.33% compared to the *GRC*, *A3C-GCN*, and *GAE-BFS*, respectively. Similarly, our proposed method achieves 26.91%, 13.15%, and 11.88% improvement in LA_Rev , compared to *GRC*, *A3C-GCN*, and *GAE-BFS*. Thanks to the upper-level agent for admission control, our proposed method can promote long-term benefits by rejecting several inappropriate VNRs in the early stage, alleviating the competition for physical resources.

3) *Resource Request Tests*: Resource-intensive VNRs tend to require more resources while others are with less resource demands. To reflect the effectiveness of VNE algorithms that embeds VNRs with various resource demands, we imitate three scenarios by adjusting the upper bound of resource requests to 50, 60, and 70.

Table VI illustrates that HRL-ACRA outperforms all the baselines in terms of the AC_Ratio and LA_Rev in this group experiment. With the increment of the upper bound of resource requests, the acceptance rates of all algorithms are declining, while the average long-term revenues are relatively stable. Our customized GNN takes full advantage of node and link features, capable of allocating resources reasonably according to the real-time situation of the physical network. The multi-objective intrinsic reward could also guide the low-level agent to achieve resource load balancing and maximize the revenue-cost ratio, which reserves more resources for subsequently arriving VNRs.

4) *Node Size Tests*: When the scale of the services is different, the number of virtual nodes of VNRs changes accordingly. For example, enterprise-level services are often extensive, while ordinary user services require few nodes. To measure the algorithm's generalizability, we mimic large, medium, and microservices by changing the node size maximum to 10, 15, and 20, respectively.

Table VII reports the experiment results in these three settings. From the results shown in Table VII, we observe that our HRL-ACRA consistently achieves the highest LA_Rev and AC_Ratio compared to other algorithms. Iteratively constructing resource allocation solutions through the seq2seq model significantly reduces the action space, speeding up training and improving generalization. Moreover, the deep feature-aware GNN encoder and positional encoder extract sufficient information on VNR's topology structure and decision order, providing more effective node representations to generate better embedded actions. It demonstrates the adaptability of HRL-ACRA to large-sized VNRs.

TABLE V
ARRIVAL RATE TESTS

Method	Arrival Rate (4/100)		Arrival Rate (6/100)		Arrival Rate (8/100)	
	AC_Ratio	LA_Rev	AC_Ratio	LA_Rev	AC_Ratio	LA_Rev
GRC	81.96 $\pm$ 2.64	22.58 $\pm$ 0.61	66.48 $\pm$ 3.04	25.39 $\pm$ 2.89	58.63 $\pm$ 2.71	27.39 $\pm$ 3.03
NRM	77.55 $\pm$ 2.05	20.71 $\pm$ 0.81	64.48 $\pm$ 2.90	23.91 $\pm$ 2.53	55.90 $\pm$ 2.01	25.48 $\pm$ 2.82
PL	86.69 $\pm$ 2.02	23.63 $\pm$ 0.62	72.64 $\pm$ 2.27	29.43 $\pm$ 3.00	63.43 $\pm$ 2.38	31.60 $\pm$ 3.50
MCTS	81.92 $\pm$ 2.14	21.12 $\pm$ 0.68	71.14 $\pm$ 2.52	25.68 $\pm$ 2.76	62.77 $\pm$ 2.24	27.90 $\pm$ 3.24
A3C-GCN	87.70 $\pm$ 1.61	24.93 $\pm$ 0.64	73.91 $\pm$ 1.92	29.49 $\pm$ 2.58	63.21 $\pm$ 2.53	30.72 $\pm$ 2.88
REINFORCE-CNN	87.49 $\pm$ 2.63	24.79 $\pm$ 0.83	72.61 $\pm$ 2.34	28.43 $\pm$ 2.72	63.68 $\pm$ 2.91	30.71 $\pm$ 2.78
GAE-BFS	88.71 $\pm$ 1.64	25.11 $\pm$ 0.75	76.00 $\pm$ 2.65	29.57 $\pm$ 3.31	66.20 $\pm$ 2.51	31.07 $\pm$ 3.05
HRL-ACRA (Ours)	91.35 $\pm$ 2.09	26.18 $\pm$ 0.73	78.84 $\pm$ 2.62	31.52 $\pm$ 2.77	69.73 $\pm$ 2.63	34.76 $\pm$ 2.83

TABLE VI
RESOURCE REQUEST TESTS

Method	Resource Request (0-50)		Resource Request (0-60)		Resource Request (0-70)	
	AC_Ratio	LA_Rev	AC_Ratio	LA_Rev	AC_Ratio	LA_Rev
GRC	81.96 $\pm$ 2.64	22.58 $\pm$ 0.61	70.67 $\pm$ 1.73	20.99 $\pm$ 0.87	61.20 $\pm$ 2.36	19.32 $\pm$ 0.76
NRM	77.55 $\pm$ 2.05	20.71 $\pm$ 0.81	66.58 $\pm$ 2.61	19.23 $\pm$ 0.64	58.84 $\pm$ 2.10	17.90 $\pm$ 0.69
PL	86.69 $\pm$ 2.02	23.63 $\pm$ 0.62	76.20 $\pm$ 2.11	23.75 $\pm$ 0.77	66.27 $\pm$ 2.45	22.01 $\pm$ 0.86
MCTS	81.92 $\pm$ 2.14	21.12 $\pm$ 0.68	72.80 $\pm$ 1.92	19.91 $\pm$ 0.61	64.25 $\pm$ 1.87	18.23 $\pm$ 0.35
A3C-GCN	87.70 $\pm$ 1.61	24.93 $\pm$ 0.64	77.19 $\pm$ 2.20	23.74 $\pm$ 0.76	67.58 $\pm$ 2.35	21.95 $\pm$ 0.78
REINFORCE-CNN	87.49 $\pm$ 2.63	24.79 $\pm$ 0.83	75.68 $\pm$ 1.98	22.90 $\pm$ 0.64	66.26 $\pm$ 2.42	21.26 $\pm$ 0.67
GAE-BFS	88.71 $\pm$ 1.64	25.11 $\pm$ 0.75	78.65 $\pm$ 1.60	23.73 $\pm$ 0.78	69.26 $\pm$ 1.83	21.83 $\pm$ 0.58
HRL-ACRA (Ours)	91.35 $\pm$ 2.09	26.18 $\pm$ 0.73	82.38 $\pm$ 2.01	25.73 $\pm$ 0.74	73.65 $\pm$ 3.28	25.62 $\pm$ 0.73

TABLE VII
NODE SIZE TESTS

Method	Node Size (2-10)		Node Size (2-15)		Node Size (2-20)	
	AC_Ratio	LA_Rev	AC_Ratio	LA_Rev	AC_Ratio	LA_Rev
GRC	81.96 $\pm$ 2.64	22.58 $\pm$ 0.61	60.42 $\pm$ 2.38	22.08 $\pm$ 0.94	48.40 $\pm$ 1.57	20.87 $\pm$ 0.69
NRM	77.55 $\pm$ 2.05	20.71 $\pm$ 0.81	57.34 $\pm$ 2.90	20.02 $\pm$ 1.10	46.25 $\pm$ 1.48	18.79 $\pm$ 0.51
PL	86.69 $\pm$ 2.02	23.63 $\pm$ 0.62	65.20 $\pm$ 2.27	25.20 $\pm$ 1.25	52.99 $\pm$ 1.10	23.78 $\pm$ 0.48
MCTS	81.92 $\pm$ 2.14	21.12 $\pm$ 0.68	60.15 $\pm$ 1.81	18.46 $\pm$ 0.61	47.47 $\pm$ 1.50	16.00 $\pm$ 0.50
A3C-GCN	87.70 $\pm$ 1.61	24.93 $\pm$ 0.64	64.83 $\pm$ 2.17	23.87 $\pm$ 0.87	51.79 $\pm$ 1.75	22.01 $\pm$ 0.42
REINFORCE-CNN	87.49 $\pm$ 2.63	24.79 $\pm$ 0.83	65.40 $\pm$ 2.77	24.36 $\pm$ 1.03	52.73 $\pm$ 1.66	22.89 $\pm$ 0.85
GAE-BFS	88.71 $\pm$ 1.64	25.11 $\pm$ 0.75	67.79 $\pm$ 2.10	24.47 $\pm$ 1.01	54.00 $\pm$ 1.54	22.20 $\pm$ 0.63
HRL-ACRA (Ours)	91.35 $\pm$ 2.09	26.18 $\pm$ 0.73	69.51 $\pm$ 2.93	26.63 $\pm$ 0.94	55.34 $\pm$ 1.58	25.58 $\pm$ 0.84

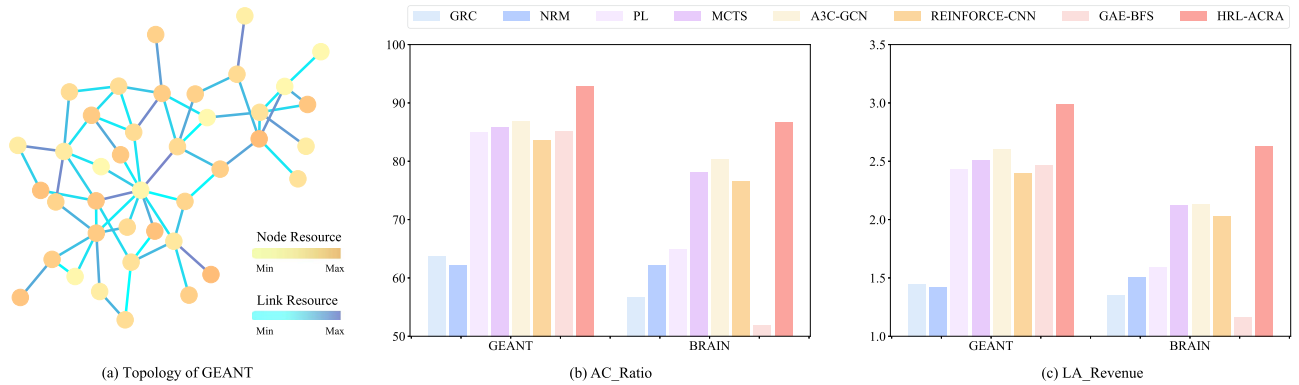


Fig. 6. Validation on real network topologies. (a) shows the topology of GEANT, where node and edge colors indicate different computing and bandwidth resource capacities, respectively. (b) and (c) presents experimental results on the acceptance ratio and long-term average revenue.

5) *Running Time Tests*: To verify the response speed of VNE algorithms, we use the average running time for tackling 1000 VNRs as the evaluation metric shown in Table IV with various node sizes. Except for MCTS based on a massive search, our proposed method and other algorithms can allocate resources

to VNRs in an acceptable time. Our proposed algorithm can process the VNR arriving online in real-time, attributing to the fast inference ability of deep neural networks. Additionally, the running time of HRL-ACRA demonstrates a minimal increase with the increase of the node size. This is due to the fact that,

as the node size increases, the failure probability of resource allocation also rises. However, HRL-ACRA is able to avoid some resource allocation processes by allowing the upper-level agent to reject non-embeddable VNRs, thus reducing overall time consumption.

C. Validation on Real Network Topologies

Similar to previous works [15], [27], we conduct additional experiments on real network topologies, GEANT and BRAIN, to verify the feasibility of our proposed algorithm when applied in realistic network environments. GEANT is the academic research network interconnecting Europe's national research and education networks, with 40 nodes and 64 edges, whose topology is shown in Fig. 6(a). BRAIN, consisting of 161 nodes and 166 edges, is the high-speed data network for scientific and cultural institutions in Berlin. Compared with GEANT with a density of 0.0821, BRAIN with a density of 0.0129 is larger but sparser. Except that the node and link resource demand of VNRs are uniformly distributed from 0 to 5, other environment simulation parameters are the same as those discussed in Section V-A. As shown in Fig. 6(b) and (c), the performance of GAE-BFS drops rapidly when applied to BRAIN, because GAE-BFS relies seriously on backtracking after failure so that it struggles to effectively explore the search space, lacking adaptation to large-scale and sparse topologies. However, HRL-ACRA still outperforms these baselines on both the acceptance ratio and long-term average revenue at both two topologies, demonstrating the robustness of applying our proposed algorithm in various real network topologies.

VI. CONCLUSION

In this article, we have introduced a novel optimization perspective and proposed a hierarchical RL framework to tackle the VNE problem. Our framework consists of two components, i.e., an upper-level agent and a lower-level agent. The upper-level agent aims to decide whether to admit the arriving VNR or not, while the lower-level tries to generate high-quality solutions of resource allocation for admitted VNRs. To address the infinite horizon problem of the upper-level agent, we have adopted the average revenue method to achieve the tradeoff of the current reward and future return. We also have designed a customized multi-objective intrinsic reward with multiple local indicators to alleviate the sparse reward issue of the lower-level agent. Moreover, leveraging the deep feature-aware GNN and the GRU-based seq2seq model, HRL-ACRA has been able to capture the temporal dependence and topological structure of VNRs and physical network. Extensive experimental results have demonstrated that our HRL-ACRA outperforms other SOTA approaches, particularly achieving significant improvement in more practical resource-limited scenarios.

In our further work, we intend to integrate more constraints (e.g., path latency and failure guarantee) into our proposed framework, which enhances the algorithm to handle more real-world network environments. We also plan to solve VNE problems with combinatorial objectives to meet the diverse indicators

of Internet providers, such as joint consideration of electricity costs.

REFERENCES

- [1] S. Yang, F. Li, S. Trajanovski, R. Yahyapour, and X. Fu, "Recent advances of resource allocation in network function virtualization," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 2, pp. 295–314, Feb. 2021.
- [2] T. Wang et al., "DRL-SFCP: Adaptive service function chains placement with deep reinforcement learning," in *Proc. IEEE Int. Conf. Commun.*, 2021, pp. 1–6.
- [3] M. Rost and S. Schmid, "On the hardness and inapproximability of virtual network embeddings," *IEEE/ACM Trans. Netw.*, vol. 28, no. 2, pp. 791–803, Apr. 2020.
- [4] N. M. M. K. Chowdhury, M. R. Rahman, and R. Boutaba, "Virtual network embedding with coordinated node and link mapping," in *Proc. IEEE Conf. Comput. Commun.*, 2009, pp. 783–791.
- [5] N. Shahriar et al., "Virtual network survivability through joint spare capacity allocation and embedding," *IEEE J. Sel. Areas Commun.*, vol. 36, no. 3, pp. 502–518, Mar. 2018.
- [6] M. Diallo, A. Quintero, and S. Pierre, "A QoS-based splitting strategy for a resource embedding across multiple cloud providers," *IEEE Trans. Serv. Comput.*, vol. 14, no. 5, pp. 1278–1291, Sep./Oct. 2021.
- [7] L. Gong, Y. Wen, Z. Zhu, and T. Lee, "Toward profit-seeking virtual network embedding algorithm via global resource capacity," in *Proc. IEEE Conf. Comput. Commun.*, 2014, pp. 1–9.
- [8] P. Zhang, H. Yao, and Y. Liu, "Virtual network embedding based on computing, network, and storage resource constraints," *IEEE Internet Things J.*, vol. 5, no. 5, pp. 3298–3304, Oct. 2018.
- [9] W. Fan, F. Xiao, X. Chen, L. Cui, and S. Yu, "Efficient virtual network embedding of cloud-based data center networks into optical networks," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 11, pp. 2793–2808, Nov. 2021.
- [10] I. Fajjari, N. Aitsaadi, G. Pujolle, and H. Zimmermann, "VNE-AC: Virtual network embedding algorithm based on ant colony metaheuristic," in *Proc. IEEE Int. Conf. Commun.*, 2011, pp. 1–6.
- [11] S. Su, Z. Zhang, A. X. Liu, X. Cheng, Y. Wang, and X. Zhao, "Energy-aware virtual network embedding," *IEEE/ACM Trans. Netw.*, vol. 22, no. 5, pp. 1607–1620, Oct. 2014.
- [12] A. Song, W.-N. Chen, Y.-J. Gong, X. Luo, and J. Zhang, "A divide-and-conquer evolutionary algorithm for large-scale virtual network embedding," *IEEE Trans. Evol. Comput.*, vol. 24, no. 3, pp. 566–580, Jun. 2020.
- [13] A. Blenk, P. Kalmbach, J. Zerwas, M. Jarschel, S. Schmid, and W. Kellerer, "NeuroViNE: A neural preprocessor for your virtual network embedding algorithm," in *Proc. IEEE Conf. Comput. Commun.*, 2018, pp. 405–413.
- [14] S. Haeri and L. Trajković, "Virtual network embedding via monte carlo tree search," *IEEE Trans. Cybern.*, vol. 48, no. 2, pp. 510–521, Feb. 2018.
- [15] Z. Yan, J. Ge, Y. Wu, L. Li, and T. Li, "Automatic virtual network embedding: A deep reinforcement learning approach with graph convolutional networks," *IEEE J. Sel. Areas Commun.*, vol. 38, no. 6, pp. 1040–1057, Jun. 2020.
- [16] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017, *arXiv:1707.06347*.
- [17] X. Ma, X. Tang, L. Xia, J. Yang, and Q. Zhao, "Average-reward reinforcement learning with trust region methods," in *Proc. Thirtieth Int. Joint Conf. Artif. Intell.*, 2021, pp. 2797–2803.
- [18] K. K. Nguyen and M. Cherié, "Environment-aware virtual slice provisioning in green cloud environment," *IEEE Trans. Serv. Comput.*, vol. 8, no. 3, pp. 507–519, May/Jun. 2015.
- [19] S. Yang, F. Li, R. Yahyapour, and X. Fu, "Delay-sensitive and availability-aware virtual network function scheduling for NFV," *IEEE Trans. Serv. Comput.*, vol. 15, no. 1, pp. 188–201, Jan./Feb. 2022.
- [20] A. Blenk, P. Kalmbach, P. van der Smagt, and W. Kellerer, "Boost online virtual network embedding: Using neural networks for admission control," in *Proc. 12th Int. Conf. Neww. Service Manage.*, 2016, pp. 10–18.
- [21] F. Habibi, M. Dolati, A. Khonsari, and M. Ghaderi, "Accelerating virtual network embedding with graph neural networks," in *Proc. Int. Conf. Netw. Service Manage.*, 2020, pp. 1–9.
- [22] Y. Xiao et al., "NFVdeep: Adaptive online service function chain deployment with deep reinforcement learning," in *Proc. IEEE/ACM Int. Symp. Qual. Serv.*, 2019, pp. 1–10.
- [23] S. Wang, J. Bi, J. Wu, A. V. Vasilakos, and Q. Fan, "VNE-TD: A virtual network embedding algorithm based on temporal-difference learning," *Comput. Netw.*, vol. 161, pp. 251–263, 2019.

- [24] M. Li and M. Lu, "A virtual network embedding algorithm based on double-layer reinforcement learning," *Comput. J.*, vol. 64, no. 6, pp. 973–989, May 2021.
- [25] H. Yao, S. Ma, J. Wang, P. Zhang, C. Jiang, and S. Guo, "A continuous-decision virtual network embedding scheme relying on reinforcement learning," *IEEE Trans. Netw. Service Manag.*, vol. 17, no. 2, pp. 864–875, Jun. 2020.
- [26] J. Cheng, Y. Wu, Y. Lin, Y. E., F. Tang, and J. Ge, "VNE-HRL: A proactive virtual network embedding algorithm based on hierarchical reinforcement learning," *IEEE Trans. Netw. Service Manag.*, vol. 18, no. 4, pp. 4075–4087, Dec. 2021.
- [27] R. Lin, L. He, S. Luo, and M. Zukerman, "Energy-aware service function chaining embedding in NFV networks," *IEEE Trans. Serv. Comput.*, vol. 16, no. 2, pp. 1158–1171, Mar./Apr. 2023.
- [28] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and P. S. Yu, "A comprehensive survey on graph neural networks," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 32, no. 1, pp. 4–24, Jan. 2021.
- [29] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. Int. Conf. Learn. Representations*, 2017.
- [30] M. Gao, B. Addis, M. Bouet, and S. Secci, "Optimal orchestration of virtual network functions," *Comput. Netw.*, vol. 142, pp. 108–127, 2018.
- [31] A. Fischer, J. F. Botero, M. T. Beck, H. de Meer, and X. Hesselbach, "Virtual network embedding: A survey," *IEEE Commun. Surveys Tuts.*, vol. 15, no. 4, pp. 1888–1906, Fourth Quarter 2013.
- [32] P. Velićković, G. Cucurull, A. Casanova, A. Romero, P. Lio, and Y. Bengio, "Graph attention networks," in *Proc. Int. Conf. Learn. Representations*, 2018.
- [33] J. Lee, I. Lee, and J. Kang, "Self-attention graph pooling," in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2019, pp. 3734–3743.
- [34] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 770–778.
- [35] M. Chen, Z. Wei, Z. Huang, B. Ding, and Y. Li, "Simple and deep graph convolutional networks," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 1725–1735.
- [36] D. Bahdanau, K. Cho, and Y. Bengio, "Neural machine translation by jointly learning to align and translate," 2014, *arXiv:1409.0473*.
- [37] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 5998–6008.
- [38] Z. Ahmed, N. Le Roux, M. Norouzi, and D. Schuurmans, "Understanding the impact of entropy on policy optimization," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 151–160.
- [39] M. Freitag and Y. Al-Onaizan, "Beam search strategies for neural machine translation," 2017, *arXiv:1702.01806*.
- [40] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," 2014, *arXiv:1412.6980*.



Tianfu Wang received the BE degree from the School of Big Data and Software Engineering, Chongqing University, in 2022. He is currently working toward the master's degree with the School of Computer Science, University of Science and Technology of China, China. His research interests include network resource management, combinatorial optimization, and data mining.



Li Shen received the PhD degree from the School of Mathematics, South China University of Technology, in 2017. He is currently a research scientist with JD Explore Academy, China. Previously, he was a research scientist with Tencent AI Lab, China. His research interests include theory and algorithms for large scale convex/nonconvex/minimax optimization problems, and their applications in statistical machine learning, deep learning, reinforcement learning, and game theory.



Qilin Fan (Member, IEEE) received the BE degree from the College of Software Engineering, Sichuan University, Chengdu, China, in 2011, and the PhD degree from the Department of Computer Science and Technology, Tsinghua University, Beijing, China, in 2017. She is currently a lecturer with the School of Big Data and Software Engineering, Chongqing University, Chongqing, China. Her research interests include network optimization, mobile edge computing and caching, network virtualization and machine learning.



WWW, AAAI, IJCAI, etc. He was the recipient of the Best Paper Award of KSEM 2020.



Tongliang Liu (Senior Member, IEEE) is the director of Sydney AI Centre with the University of Sydney. He is broadly interested in the fields of trustworthy machine learning and its interdisciplinary applications, with a particular emphasis on learning with noisy labels, adversarial learning, transfer learning, unsupervised learning, and statistical deep learning theory. He has authored and co-authored more than 200 research articles including ICML, NeurIPS, ICLR, CVPR, AAAI, IJCAI, *Journal of Machine Learning Research*, and *IEEE Transactions on Pattern Analysis and Machine Intelligence*. His monograph on machine learning with noisy labels will be published by MIT Press. He is/was a (senior-) meta reviewer for many conferences, such as ICML, NeurIPS, ICLR, UAI, AAAI, IJCAI, and KDD, and was a notable AC for ICLR. He is an associate editor of *Transactions on Machine Learning Research* and is on the editorial boards of *Journal of Machine Learning Research* and MLJ. He is a recipient of the AI's 10 to Watch Award from IEEE, in 2023, the Future Fellowship Award from Australian Research Council (ARC), in 2022, and the Discovery Early Career Researcher Award (DECRA) from ARC, in 2018.



Hui Xiong (Fellow, IEEE) received the PhD degree in computer science from the University of Minnesota. He is currently a chair professor with the Hong Kong University of Science and Technology (Guangzhou). His research interests include data mining, mobile computing, and their applications in business. He has served regularly on the organization and program committees of numerous conferences, including as a program co-chair of the Industrial and Government Track for the 18th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD), a program co-chair for IEEE 2013 International Conference on Data Mining (ICDM), a general co-chair for the 2015 IEEE International Conference on Data Mining (ICDM), and a program co-chair of the Research Track for the 2018 ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. He received the 2021 AAAI Best Paper Award and the 2011 IEEE ICDM Best Research Paper award. For his outstanding contributions to data mining and mobile computing, he was elected an AAAS fellow, in 2020.